Show me the complete 16-domain audit table

Detail the deployment and feature flag rollback plan
List all material artifacts needed for domain approval
Show the exact validation tests for each green status
Explain the shared blockers across all sixteen domains
Sources
62
provide everything needed and all documents artfacts lines of code documents matracies diagrams needed to take eveything there green

I created a downloadable **completion** kit containing the complete 16-domain audit snapshot, approval-artifact matrix, validation-test matrix, deployment and rollback runbook, diagrams, domain completion contracts, and reference evidence-gate code.

The kit contains:

- 16 domain completion contracts.
- 416 proposed validation cases: 20 shared green-status checks applied to each domain, plus 6 domain-specific cases per domain.
- 256 approval-artifact requirements: 16 required artifact categories across 16 domains.
- 28 feature-flag rollback entries enumerated from the retrieved source registry.
- Three architecture, release-state, and rollback diagrams.
- Reference Python gate code, a test module, and an unexecuted SQL evidence-metadata schema.
- Release-manifest, acceptance, work-package, and operated-drill templates.

The ZIP contains 36 files. The main blueprint and CSV matrices are also supplied separately.

Important boundary: these are execution materials and reference code—not completed production implementation. I ran 15 local assertions against the reference evidence checker; all passed. I did not run Semester's application tests, deploy code, apply migrations, enable flags, or obtain institutional/professional approvals.

1. Complete 16-domain audit table

This table preserves the screenshot's six maturity facts. It is a snapshot reconciliation—not a fresh audit of the running deployment. All 16 "Designed" entries represent prototypes, not accepted, production-complete designs. [1]

Domain	Designed	Implemented	Tested	Deployed	Institution-authorized	Production-enabled	Screenshot packages
Identity and onboarding	Prototype	Partial	Account-sync required check	Hosted PWA, signed-in sync	No	No	3

Domain	Designed	Implemented	Tested	Deployed	Institution-authorized	Production-enabled	Screenshot packages
Planning and degree	Prototype	Partial	Not evidenced	Hosted PWA, seed data	No	No	5
Registration	Prototype	Server code, not release-ready	Server tests, per register §3	No tenant	No	No	6
Courses and teaching	Prototype	Partial	Not evidenced	Hosted PWA, personal courses	No	No	5
Assignments and work	Prototype	Partial; per-row engine only for tasks	Calendar keyboard and open-calendar tests	Hosted PWA, device-first	No	No	5
Study and tutoring	Prototype	Client-only/mock and partial	Not evidenced	Hosted PWA	No	No	9
Grades and progress	Prototype	Server code, not release-ready	Server tests	No tenant	No	No	5
Advising and support	Prototype	Partial	Not evidenced	No staffed route	No	No	6
Career and portfolio	Prototype	Client-only/mock	Not evidenced	Hosted PWA, local	No	No	7
Campus and community	Prototype	Partial	`Community.test.tsx`	Hosted PWA	No	No	7
Housing, dining, services	Prototype	Partial	Not evidenced	Hosted PWA	No	No	7
Money and finance	Prototype	Partial	Billing live-acceptance evidence	Checkout held	No	No	8
Institution OS	Prototype	Server code, not release-ready	Server tests	No tenant; gateway configuration dependency	No	No	6
Company OS	Prototype	Documents	—	—	No	No	8
Developer and partner	Prototype	None found	—	—	No	No	10
Governed AI	Prototype	Partial, ungoverned	Not gated	Hosted shared-key function	No	No	9

The package counts above are screenshot counts. They are not a recovered inventory of named tasks. The kit's proposed packages must be reconciled with the original work-package register rather than treated as those exact existing packages.[1]

Material discrepancy: 21 vs. 28 flags

The screenshot and historical live-status report say 21 flags. The `app/src/lib/flags.ts` content retrieved during this turn enumerates 28 entries:

Category	Entries
Modules	7
Temporary release	1
Connectors	7
Provider scopes	3
Writebacks	3
Operations	3
Safety	3
Experiment	1
Total	28

This means the completion register needs regeneration. It does not mean seven more features are complete. The retrieved source states that flags default off; its returned evaluator content was truncated, so I have not reverified the entire evaluator implementation.

2. What each green status must prove

Do not calculate green from “code exists,” “a test file exists,” or “a page is hosted.”

Status	Green condition	Blocking condition
Designed	Required journeys, roles, states, responsive behavior, accessibility, and consequences accepted	Missing critical journey or undefined authority
Implemented	Runtime-connected functionality with durable state, authorization, provenance, audit, and error handling	Required mock, unwired service, insecure entry point, or incomplete persistence
Tested	Applicable tests actually executed against the candidate; qualified reviews accepted	Missing runner, skipped test, stale run, or unresolved release blocker
Deployed	Actual release, configuration, functions, workers, and database match approved target	Wrong build, schema drift, unwired service, or unverified target
Institution-authorized	Current acceptance for the precise institution, data, operations, roles, and cohort	Missing, expired, withdrawn, or mismatched authority
Production-enabled	Scoped activation with operating staff, alerts, recovery, reconciliation, and accepted stop criteria	Unstaffed service, untested kill switch, missing recovery, or unauthorized expansion

For company-internal or purely personal capabilities, institutional authorization may be legitimately not applicable. Record an accepted rationale; do not invent an institutional approval.

The project’s authority-gate document goes further for system replacement: it requires functional parity, data authority, accessibility, security, privacy, isolation, auditability, migration integrity, reconciliation, dual-run reliability, institutional approval, support, rollback, exportability, commercial readiness, and outcome proof. [2]

3. Exact shared validation tests for green status

The test matrix gives each case a domain-prefixed ID, procedure, expected result, evidence requirement, and owner role. Every application case currently says `NOT RUN`, and every runner binding says `UNBOUND`.

These are exact acceptance assertions to implement—not fabricated existing test commands.

Designed

Test	Procedure	Required result
`G-DES-01`	Review every required journey and role against accepted scope	All required states and actions exist; omissions remain held
`G-DES-02`	Review keyboard, focus, errors, responsive behavior, and consequence designs	Accessible behavior and consequential-action semantics are specified and accepted

Implemented

Test	Procedure	Required result
`G-IMP-01`	Trace each mutation from UI through authorization to durable storage	Required functionality is runtime-connected; receipts and audit are wired
`G-IMP-02`	Forge tenant, owner, identity, and scope at every entry point	Denial without protected metadata exposure or mutation
`G-IMP-03`	Restart, refresh, and read committed data across devices	Canonical state, version, and provenance survive

Tested

Test	Procedure	Required result
`G-TST-01`	Execute all applicable candidate-bound tests	Every blocking case passes; missing or skipped execution is not a pass
`G-TST-02`	Qualified keyboard, assistive-technology, and reflow evaluation	Applicable criteria accepted after remediation and retest
`G-TST-03`	Independent security assessment and blocker rescan	No open launch-blocking finding

Use a defined evaluation scope and representative complete journeys, not a single automated accessibility score. W3C's evaluation methodology explicitly includes scoping, exploring assets, selecting samples, evaluating them, and reporting findings. [3]

Deployed

Test	Procedure	Required result
`G-DEP-01`	Read actual build, deployment, configuration, and schema fingerprints	They match the approved candidate and target
`G-DEP-02`	Exercise permitted and prohibited journeys on the actual target	Persistence, denials, callbacks, and queues work correctly

Test	Procedure	Required result
G-DEP-03	Test previous and new app/function versions against expanded schema	The planned rollback is compatible

Institution-authorized

Test	Procedure	Required result
G-AUT-01	Validate approval scope, reviewers, target, cohort, data, and operations	All applicable authority is current and scope-matched
G-AUT-02	Expire or withdraw an approval and attempt activation	Activation fails closed
G-AUT-03	Run customer UAT and inspect acceptance	Authorized customer reviewer accepts exact scope and limitations

Production-enabled

Test	Procedure	Required result
G-PRD-01	Attempt use outside the approved tenant, cohort, or operation	Server, worker, and direct API deny it
G-PRD-02	Kill each active high-risk feature; attempt fresh dispatch	Dispatch stops within the accepted, measured propagation bound
G-PRD-03	Restore isolated database, matching objects, and configuration	Checksums, policies, provenance, and application reads pass
G-PRD-04	Exercise alert/support response with primary unavailable	Backup acknowledgement and escalation work
G-PRD-05	Exercise export, deletion, revocation, and offboarding	Complete policy-correct result, including legal holds
G-PRD-06	Observe the agreed operating window and reconcile outcomes	Accepted SLO, error-budget, outcome, and stop policies are met

Numerical thresholds must be approved for the real service. The kit does not invent recovery commitments, performance guarantees, or customer response SLAs.

4. Domain-specific validation suite

Each row below has six concrete cases in the downloadable matrix and its domain contract. These supplement—not replace—the shared green-status tests.

Domain	Six required domain cases
Identity and onboarding	Cross-device account persistence; forged owner/tenant denial; SSO/SCIM deactivation; export/erasure with holds; interrupted-sync retry; family-share revocation
Planning and degree	Nested prerequisite evaluation; catalog-year change; transfer/substitution; schedule/workload conflicts; record-access revocation; official-audit reconciliation
Registration	Last-seat race; idempotent replay; timeout after source commit; eligibility changes before commit; swap failure; kill switch with queued commands

Domain	Six required domain cases
Courses and teaching	Draft versus published visibility; roster removal; conflicting syllabus extraction; instructor/TA permissions; unsupported import elements; archive/export
Assignments and work	Canonical deadline propagation; interrupted submission/upload; offline edit conflicts; stable nested-tab links; selected-document sharing; tree keyboard behavior
Study and tutoring	Source-grounded answers; retrieved prompt injection; course AI disablement; booking race; time-zone-aware rescheduling; staffed tutoring escalation
Grades and progress	Grading-policy fixtures; unauthorized release; duplicate/out-of-order passback; correction history; private versus official labels; previous-version compatibility
Advising and support	Caseload denial; consent expiry; reassignment; unavailable-primary escalation; sensitive-record exclusion; referral closeout/export
Career and portfolio	Selective publication; employer-share revocation; posting review; export fidelity; credential tamper rejection; mentor/employer deauthorization
Campus and community	Nonmember denial; block consistency; report escalation; appeals; event changes; safe pseudonymity rollback
Housing, dining, services	Allocation race; duplicate order prevention; provider uncertainty; kill with legitimate refund path; residence privacy; database-plus-file restore
Money and finance	Duplicate/out-of-order webhooks; failed-renewal recovery; refund/dispute reconciliation; financial-owner forgery; provider/ERP reconciliation; unauthorized-checkout denial
Institution OS	Provision/deprovision; two-person high-impact approval; partial workflow retry; dual-run reconciliation; safe Core hold; offboarding
Company OS	Changed quote requires reapproval; contract/invoice/payment reconciliation; company-role student-record denial; actual monthly close; backup paging; unsupported-claim hold
Developer and partner	Invalid token/scope denial; webhook forgery/replay; sandbox-production separation; key revocation; version compatibility; expanded-scope approval
Governed AI	All-entry-point gateway enforcement; restricted-data denial; injection cannot mutate records; budget races; global/tenant kill; invalid citation handling

Example: registration test with real assertions

For `D03-FLOW-03`, “timeout after authoritative source commit”:

1. Seed an eligible synthetic student and available section.
2. Submit with a stable transaction/idempotency identifier.
3. Have the approved test adapter commit the enrollment.
4. Inject a timeout before Semester receives the receipt.
5. Retry or invoke reconciliation.
6. Assert one authoritative enrollment exists.
7. Assert Semester reports an uncertain/pending state until reconciliation.
8. Assert no second irreversible command is issued.
9. Assert the final receipt names the authoritative transaction.
10. Retain source-state, application-state, correlation, and audit evidence.

The fixture adapter, actual service calls, and assertion bindings still need to be connected to the repository’s real interfaces.

5. All material artifacts needed for domain approval

The project already specifies 16 artifact categories for every domain. The kit expands them into 256 trackable domain-artifact requirements. [2]

Artifact	Required material	Acceptance authority
Product model	Capability boundaries, critical journeys, parity, inclusions/exclusions	Product and domain authority
User screens	Roles, states, responsive behavior, accessibility, consequence copy	Design/accessibility
Data model	Canonical schema, authority, provenance, classification, retention, portability	Data/Privacy
Workflows	Commands, events, retries, approvals, state machines, compensation	Engineering/domain authority
Policy controls	Policy hierarchy, runtime enforcement, holds, kill-switch behavior	Security/Privacy
Roles and capabilities	Least privilege, consent, scopes, expiry, revocation	Security/customer IT
Audit	Event contract, integrity, permitted readers, retention	Security/domain auditor
Support process	Queue, named staff/backup, hours, escalation, articles, drills	Support
Monitoring/SLO	Metrics, traces, sanitized logs, alerts, accepted targets	Reliability
Documentation	Student, staff, admin, developer, and operator guidance	Product/Operations
Test plan	Fixtures, runner bindings, assertions, negative/concurrency/recovery cases	QA/reviewers
Release gate	Exact candidate, deployment, configuration, schema, cohort, authority	Release authority
Migration/rollback	Expand-contract, dual run, cutover, checksums, restoration, compensation	Engineering/customer IT
Commercial model	Applicable pricing, signing, tax, payments, insurance, agreements	Finance/counsel
Customer-success model	Onboarding, training, baseline, outcomes, closeout, reference permissions	Success/customer sponsor
Operating owner	Named accountable, implementation, acceptance, and backup personnel	Founder/Operations

Domain-specific supplements

Domain	Additional materials
Identity	Account lifecycle, SSO mapping, provisioning rules, consent/retention schedule
Planning	Versioned catalogs, requirements fixtures, transfer/equivalency and exception rules
Registration	SIS contract, transaction state machine, override authority, uncertainty handling
Courses	Publishing rules, syllabus fixtures, course AI policy, enrollment-access model
Assignments	Submission states, deadline/time-zone policy, document/file schema, offline policy
Study/tutoring	Source/evaluation fixtures, tutor qualification policy, booking and escalation rules
Grades	Grading fixtures, release authority, passback mapping, correction/regrade procedures
Advising	Caseload/consent matrix, referral taxonomy, sensitive exclusions, staffing rota
Career	Portfolio schema, posting policy, credential authority, employer/mentor controls
Community	Moderation policy, appeals, safety escalation, source ownership

Domain	Additional materials
Services	Provider mappings, allocation rules, entitlements, fulfillment/refund responsibilities
Finance	Money-flow map, price book, tax/accounting advice, processor/ledger controls
Institution OS	Tenant map, migration/cutover, customer UAT, responsibility matrix
Company OS	Entity/signing evidence, actual finance records, agreements, insurance and staffing
Developer/partner	API contract, scopes, webhook specification, sample client, review and partner terms
AI	Call-site inventory, provider/data rules, model/tool policy, evaluations, red-team, incident ladder

The templates do not substitute for executed agreements, independent assessments, actual financial records, or customer signatures.

6. Deployment plan

A. Establish the actual deployment topology

Do this before selecting rollback commands:

```
Commit
  ├─ Page/application host
  ├─ Server/API host
  ├─ Supabase Edge Functions
  ├─ Workers and scheduled jobs
  ├─ Database migrations
  ├─ File/object storage
  └─ Runtime policy and feature flags
```

The retrieved `ROLLBACK.md` describes GitHub Pages and independently deployed Edge Functions. Therefore, a Vercel rollback must not be assumed to roll back the canonical Semester page.

Also inspect external deployment automation. The repository rollback document describes Supabase Branching applying migrations independently of repository workflow commands and contains historical repair notes. Live settings and the migration ledger must resolve that history.

B. Bind the release to an immutable manifest

Record:

- Commit SHA and build digest.
- Hosting provider, project, and deployment ID.
- Database project and region.
- Schema fingerprint and migration checksums.
- Function and worker versions.
- Build-time environment fingerprint—without secrets.
- Runtime policy/configuration version.
- Flag snapshot version.
- Tenant, cohort, and permitted operations.

- Evidence and approver references.
- Predecessor release.
- Rollback compatibility.
- Expiry/revalidation conditions.

C. Deploy dark

1. Rehearse additive schema changes.
2. Test the prior application against the expanded schema.
3. Keep new features disabled.
4. Deploy application, functions, and workers.
5. Run actual-target smoke and denial tests.
6. Verify callbacks, queues, reconciliation, alerts, and support coverage.
7. Obtain scope-matched activation acceptance.
8. Enable only the approved cohort.

Advancement should depend on accepted observations and thresholds, not arbitrary rollout percentages.

7. Feature-flag rollback plan

The kit includes every enumerated flag and its proposed safe rollback. This summary groups related entries.

Flag group	Primary stop mechanism	Preserve/reconcile
`module.core_mode`	`kill.core_modules`	Freeze native data; do not delete it or silently revert to stale external authority
Integration dashboard	Tenant/release policy off	Read-only records and audit
Institutional operations	Build-time flag off and redeploy	Local drafts and export provenance
Source freshness cards	Scoped disable/connection hold	Source labels and stale-data disclosure
Campaign manager	Policy off / sharing kill	Sent-message audit; scheduled-send cancellation
Sponsorship	Policy off / sharing kill	Permitted audit; no new disclosures
Dining	Writeback/integration kill	Existing orders, charges, authorized refunds/cancellations
Seven connectors	Connection/integration kill	Lineage, pending runs, reconciliation, credential revocation
Three read scopes	Withdraw scope	Permitted mapped references; stop refreshing
Three writebacks	Writeback/integration kill	In-flight authoritative outcomes and approved compensation
External AI	AI kill	Private permitted audit and budget reconciliation
Uploads	Upload kill	Committed objects; quarantine pending scan
Code execution	Execution kill	Job evidence; approved termination
Pseudonymity	Hold affected surface	No unintended identity disclosure
Volunteer moderation	Remove privilege	Transfer cases to staffed operators

Flag group	Primary stop mechanism	Preserve/reconcile
Institution escalation	Safe staffed handoff	Urgent escalation route and acknowledgements
Ranking experiment	Experiment off	Canonical tasks; previous ranking

Two registry behaviors require review before activation

The retrieved registry describes:

- Core rollback as returning modules to Connect.
- Pseudonymity rollback as falling back to names.

Those behaviors must not be applied blindly.

If Core has become authoritative, returning to an unreconciled external source can present incorrect records. If pseudonymity was promised, reverting to names can disclose identities. The proposed runbook therefore uses held/read-only behavior until the appropriate domain/privacy authority accepts the transition.

8. Rollback procedure

Immediate containment

1. Declare the incident and affected scope.
2. Capture release, policy, flag, and schema versions.
3. Engage the server-enforced kill for affected operations.
4. Verify direct API and worker dispatch denial.
5. Pause producers/consumers and checkpoint pending transaction IDs.
6. Quarantine uncertain outcomes; do not delete queues or receipts.

Restore compatible application behavior

7. Disable the affected scoped rollout.
8. Invalidate policy caches.
9. Restore the known-good application/page.
10. Restore affected functions and workers separately.
11. Confirm the old application still honors the current kill state.
12. Verify service-worker and client-version behavior without wiping private drafts.

Vercel’s Instant Rollback restores a previous deployment, and environment-variable changes apply to new deployments rather than previous builds. An older deployment must therefore be assessed against the current service dependencies and security state. [4] [5]

Repair data and external effects

13. Reconcile external source outcomes before replay.
14. Preserve committed registration, grade, and financial records.
15. Use approved correction/compensation—not arbitrary database reversal.
16. Prefer forward schema repair when compatible.
17. If restoration is necessary, restore into an isolated target first.
18. Reconcile database state, object bytes, permissions, and post-backup changes.

Supabase database backups include storage metadata but not the actual stored file objects. A complete file recovery requires matching object-byte backups and checksum verification.[6]

Reauthorize service

19. Run smoke, denial, recovery, privacy, and domain regressions.
20. Confirm alerts and support coverage.
21. Record measured detection, kill propagation, restoration, and reconciliation.
22. Obtain fresh acceptance.
23. Reactivate only the approved scope.

A flag rollback cannot recall an email, undo an already accepted enrollment, reverse a published grade automatically, or restore a deleted file's bytes.

9. Shared blockers across all sixteen domains

These are inherited blockers. Some directly affect every domain; others affect all domains using the relevant shared service.

Blocker	Effect	Closure evidence	Owner
Unverified candidate/target	No dependable deployed status	Candidate manifest and actual-target fingerprints	Engineering/Release
Incomplete authorization coverage	UI checks can be bypassed	Every-entry-point denial suite	Security/Backend
Local/blob/duplicate records	Durable canonical state is unclear	Entity migration, authority mapping, conflict/recovery tests	Data/Product
Unproven adapters/async execution	Connected workflows cannot reconcile reliably	Contract tests, outbox/DLQ, operated adapter evidence	Integrations
AI bypasses	Policy, budget, and kill controls can be evaded	All-call-site enforcement and broker/evaluation tests	AI/Security
Missing qualified accessibility acceptance	Automated tests cannot establish complete accessibility	Manual evaluation, remediation, retest	Accessibility
Unstaffed support/paging	A deployed service is not operated	Named primary/backup and acknowledgement drills	Operations
Unproved recovery/privacy operation	Restore or revocation may fail in practice	Candidate-bound restore/rollback/rights/offboarding drills	Reliability/Privacy

Blocker	Effect	Closure evidence	Owner
Missing institutional/commercial authority	Code cannot authorize official use or sales	Accepted agreements, customer UAT, signing/payment authority	Founder/counsel/customer
Stale/conflicting registers	Dashboard can falsely imply completeness	Generated inventory, timestamps, conflict resolution	Product/Engineering

The project's master gap register also identifies workflow gaps in reconciliation, consented sharing, launch readiness, monitoring, support routing, incident command, cutover, rollback windows, partner review, and outcome measurement. These are design-catalog findings, not fresh runtime defects, but they must be reconciled rather than ignored. [7]

10. Code and schema supplied—and their limits

The ZIP includes a working offline evidence-consistency helper.

It rejects:

- Missing evidence.
- Duplicate/ambiguous evidence.
- Failed or unaccepted evidence.
- Withdrawn approvals.
- Expired or future-dated evidence.
- Wrong tenant/scope.
- Wrong operation.
- Wrong deployment/configuration/schema.
- Missing approver.
- Unresolved target.
- An engaged kill switch.

The 15 executed assertions validate this helper's logic only.

Integration requirements

Do not install it as a second authorization system.

Extend Semester's existing:

- Release-profile evaluator.
- Activation receipt/contract handling.
- Feature-flag evaluator.
- Server-side policy enforcement.
- Audit and evidence storage.

At a production boundary, evidence must come from an authenticated, controlled store—not request-body booleans. Approval provenance, reviewer authority, expiry, withdrawal, and concurrent revocation require enforcement.

The supplied SQL is a proposed evidence-metadata schema. It was not applied. It deliberately does not grant browser access; production policies, runtime roles, immutable audit, and trusted approval ingestion still require implementation and review.

Why I have not supplied a fictional line count

The amount of application code needed cannot be grounded from the screenshot. Existing implementation must first be traced to required capabilities, reused where correct, and patched where incomplete.

A large amount of generated code would not prove:

- Correct institutional rules.
- Adapter compatibility.
- Tenant isolation.
- Successful recovery.
- Staff availability.
- Executed authority.

The kit provides reviewable contracts and code foundations without pretending that the remaining domain implementation has already been written.

11. Preserve full scope and long-term authority gates

The 16 rows are an aggregation—not permission to narrow Semester.

Assessment, academic records, family/guardian, integration/migration, data graph, operations, and broader institutional functions need explicit sub-capability packages inside the relevant domains. The project's native catalog contains a broader domain structure than the screenshot. [8]

Some designed authority gates also require observation over a full term, a registration cycle, or two terms. Examples include gradebook dual run, academic-record reconciliation, assessment reliability, advising operations, finance reconciliation, and moderation performance. These are proposed project gates, not already achieved evidence, and cannot honestly be completed in a same-day code push. [2]

12. Execution order to take the register green

1. Reconcile the 16-row dashboard with the detailed capability and flag inventories.
2. Identify actual deployment/database targets and pin a candidate.
3. Assign named owners, backups, and acceptance authorities.
4. Resolve scope, source authority, and conflicting commercial/customer claims.
5. Close shared authorization, persistence, AI, asynchronous, and operational blockers.
6. Implement missing domain journeys against canonical objects.
7. Bind the 416 proposed cases to real fixtures, services, and retained runs.
8. Execute verification and qualified independent reviews.
9. Deploy dark and verify the actual target.
10. Obtain applicable institution/legal/commercial acceptance.
11. Activate the bounded, authorized cohort.

12. Complete required operated and dual-run observation.
13. Accept each maturity cell independently.
14. Permit only claims supported by current evidence.

The materials are ready as a completion-program kit. The application tests remain unexecuted, approval records remain unsigned/unverified, and production activation remains held until those real conditions are satisfied.

⁂

1. Screenshot-2026-10-11-at-3.57.31-PM.jpg
2. The Main Semester design system (2) copy/docs/native-platform/NATIVE_AUTHORITY_GATES.md
3. https://www.w3.org/WAI/test-evaluate/conformance/wcag-em/
4. https://vercel.com/docs/instant-rollback
5. https://vercel.com/docs/environment-variables
6. https://supabase.com/docs/guides/platform/backups
7. The Main Semester design system (2) copy/docs/master/SEMESTER_GAP_REGISTER.md
8. The Main Semester design system (2) copy/docs/native-platform/NATIVE_DOMAIN_CATALOG.md
9. The Main Semester design system (2) copy/docs/master/SEMESTER_DATA_DOMAIN_CATALOG.md
10. https://www.w3.org/WAI/test-evaluate/tools/list/
11. https://www.w3.org/WAI/test-evaluate/preliminary/
12. https://www.w3.org/WAI/
13. https://www.w3.org/WAI/eval/report-tool/
14. https://www.w3.org/WAI/ER/2011/eval/Overview.html
15. https://www.w3.org/WAI/planning/statements/generator/
16. https://supabase.com/docs/guides/platform/migrating-within-supabase/backup-restore
17. https://vercel.com/docs/rolling-releases
18. https://simplebackups.com/blog/restore-supabase-storage-objects
19. https://revivedb.dev/blog/do-supabase-backups-include-storage
20. https://www.w3.org/WAI/test-evaluate/
21. The Main Semester design system (2) copy/docs/native-platform/NATIVE_GAP_REGISTER.md